

Projeto — MovieManagement

Arquitetura em Camadas + Persistência + Git

1. Objetivo Geral

Desenvolver uma aplicação em C# utilizando:

- Arquitetura em Camadas
- Interfaces
- Regras de Negócio
- Persistência de Dados
- Git e GitHub

O projeto deverá evoluir ao longo de várias fases, permitindo consolidar progressivamente os conceitos abordados nas aulas.

2. Regras Gerais do Projeto

• Repositório GitHub

Cada aluno deverá:

- criar um repositório público no GitHub logo no início do projeto
- enviar o link do repositório ao formador - anexar na tarefa em bloco de notas

O objetivo é permitir:

- acompanhamento da evolução do projeto desde o início
- verificação da utilização do Git
- análise dos commits realizados

• Commits obrigatórios

No final de cada parte deverá existir pelo menos: 1 commit identificando a conclusão dessa fase.

Exemplo:

```
Conclusão Parte 1  
Conclusão Parte 2  
Conclusão Parte 3
```

• Boas práticas obrigatórias

O projeto deverá demonstrar:

- organização do código
- nomes claros de classes e métodos
- separação de responsabilidades

- redução de código duplicado
- comentários quando necessário
- arquitetura consistente
- utilização correta das camadas

- **Arquitetura obrigatória**

A aplicação deverá manter:

Camada	Responsabilidade
UI	interação com utilizador
Business	regras de negócio
Data	persistência
Domain	entidades e interfaces

- **Estrutura esperada**

```
MovieManagement
|
├─ MovieManagement.UI
├─ MovieManagement.Business
├─ MovieManagement.Data
└─ MovieManagement.Domain
```

3. Entrega Final

Formato: ficheiro .zip, com solution completa e código funcional

Data limite : 8 de Junho

Parte 1 – Implementação da entidade Filme

1. Entidade Filme

Cada filme deverá possuir:

- Id
- Título
- Ano
- Língua
- Classificação

2. Funcionalidades obrigatórias

- adicionar filme
- listar filmes
- procurar filme por título
- remover filme

3. Regras de negócio

Título

- obrigatório
- não pode existir duplicado

Classificação

- deve estar entre 0 e 5

4. Persistência

Nesta fase:

- persistência em memória (List<Filme>)

5. Commit obrigatório: Conclusão Parte 1

Parte 2 – Implementação das entidades Categoria e Realizadores

1. Entidade Categoria

Cada categoria deverá possuir:

- Id
- Nome

2. Entidade Realizador

Cada realizador deverá possuir:

- Id
- Nome
- País

3. Funcionalidades obrigatórias

Categorias

- adicionar categoria
- listar categorias
- procurar categoria
- remover categoria

Realizadores

- adicionar realizador
- listar realizadores
- procurar realizador
- remover realizador

4. Regas de Negócio

Categoria

- nome obrigatório
- não permitir categorias duplicadas

Realizador

- nome obrigatório
- país obrigatório

5. Persistência

Nesta fase:

- persistência em memória (List<Categoria>)
- persistência em memória (List<Realizador>)

6. Commit obrigatório: Conclusão Parte 2

Parte 3 – Relação entre Entidades + SQLite

Nesta fase o sistema deverá evoluir para suportar:

- relações entre entidades
- persistência em SQLite

2. Relações Obrigatórias

Cada filme deverá possuir:

- uma categoria
- um realizador

3. Atualização da entidade Filme

A entidade Filme deverá passar a incluir:

```
CategoriaId  
RealizadorId
```

- **Regras importantes**

Antes de adicionar um filme:

- a categoria deve existir
- o realizador deve existir

Estas validações devem ocorrer na Business Layer

4. Persistência obrigatória

O sistema deverá possuir simultaneamente:

- ✓ Persistência em memória utilizando List<T>
- ✓ Persistência em SQLite utilizando SQLite

IMPORTANTE: A arquitetura deverá permitir trocar a persistência **sem alterar:** UI, Business e Domain.

5. Commit obrigatório: Conclusão Parte 3

Questões de reflexão

1. Porque é importante separar responsabilidades pelas camadas?
2. Qual a vantagem das interfaces?
3. O que mudou ao introduzir SQLite?
4. Porque é importante validar relações entre entidades?
5. Que vantagens trouxe a arquitetura utilizada?
6. Que melhorias poderiam ser feitas no código?

Critérios de Avaliação

1. Estrutura e Arquitetura (camadas) – 20%

2. Regras de Negócio e Lógica Aplicacional – 20%

3. Persistência de Dados (List<T> + SQLite) – 20%

4. Git e GitHub – 15%

- Criação e utilização de repositório público desde o início do projeto.
- Commits realizados ao longo do desenvolvimento, com mensagens claras e associadas às partes do enunciado.
- Histórico que evidencie evolução progressiva do projeto

5. Qualidade do Código e Boas Práticas – 15%

6. Implementações Adicionais / Extensões – 10%

- Funcionalidades extra para além do enunciado (por exemplo: filtros, ordenações, pesquisa avançada, menus mais completos, relatórios simples, etc.).
- Melhorias de usabilidade ou de organização da aplicação que demonstrem iniciativa e curiosidade técnica.
- Exploração de conceitos não obrigatórios, mas coerentes com a arquitetura proposta.